

Project 2 보고서

이상협(2021-11099) 박상윤(2024-12879)

1. 서론

이번 프로젝트는 통계 및 머신러닝 분야의 텍스트 데이터를 대상으로 데이터마이닝의 핵심 과업을 수행하는 것을 목적으로 하며, 크게 세 부분으로 나뉜다. PART I에서는 Q&A 커뮤니티의 질문-태그 데이터를 분석하여 태그 사이의 연관 관계를 규명하고자 하였고, PART II에서는 학술 논문 집합에 대하여 자연어 질의어로 관련 문서를 찾아 주는 정보 검색 엔진을 구축하고 그 검색 성능을 개선하고자 하였다. PART III에서는 네 개 학술 저널의 논문 본문을 학습하여 어느 저널의 논문인지 분류하는 텍스트 분류 모델을 구축하였다.

세 과업은 서로 독립적이므로 각각 AA, SE, CL 폴더로 코드를 분리하여 작성하였으며, 과제에서 요구한 산출물 파일명 규칙(DMA_project2_team10_*, 팀 번호 10)을 따르도록 하였다. 각 폴더는 그 안에서 스크립트 하나만 실행하면 결과 파일(.pk1 이나 인덱스)이 다시 만들어지고 성능도 재측정되도록 구성하여, 채점 시 재현이 용이하도록 하였다. 특히 PART II 검색 엔진은 특정 질의어나 특정 문서에 대한 예외 처리 없이 모든 질의어에 동일하게 동작하는 범용 로직으로만 구현하였으며, 정답 파일(relevance.txt)은 코드 어디에서도 참조하지 않고 오직 문서 집합 자체의 통계적 특성만을 활용하였음을 먼저 밝혀 둔다. 이는 과제 주의사항에서 요구한 일반성 조건을 위반하지 않기 위한 것이다.

이하에서는 각 파트별로 코드의 구조를 먼저 설명하고, 성능 또는 적합성을 높이기 위하여 어떤 시도를 하였으며 그중 어떤 방법이 실제로 효과가 있었고 어떤 방법은 오히려 효과가 없었는지를 실험 결과와 함께 서술한다.

2. PART I. 연관 분석

PART I에서는 CrossValidated 와 같은 통계·머신러닝 Q&A 커뮤니티에 게시된 질문들과 거기에 달린 태그 데이터(DMA_project_UBR.csv)를 분석 대상으로 하여, 어떤 태그들이

함께 등장하는 경향이 있는지를 연관 분석으로 규명하고자 하였다. 코드는 AA/part1.py 한 파일에서 데이터 로드, 수평 테이블 생성, 빈발 아이템셋 및 연관 규칙 도출을 순서대로 수행하도록 작성하였다.

(1) 데이터 로드 및 horizontal table 생성

먼저 pandas 로 DMA_project_UBR.csv 를 읽어 들였다. 이 파일은 질문(question)과 태그(tag) 두 열로 구성된 긴(long) 형태의 데이터로, 총 117,656 개의 질문-태그 레코드가 들어 있으며 고유 질문은 42,921 개, 고유 태그는 1,032 개였다. 연관 분석을 수행하려면 각 질문을 하나의 transaction 으로, 각 태그를 item 으로 보는 수평 테이블이 필요하므로, pd.crosstab(df["question"], df["tag"])를 사용하여 질문 id 를 행으로, 태그 이름을 열로 하는 (42,921 × 1,032) 크기의 원-핫 테이블을 생성하였다.

```
df = pd.read_csv("DMA_project_UBR.csv")
horizontal = pd.crosstab(df["question"], df["tag"]).astype(bool)
```

이때 astype(bool)로 자료형을 불리언으로 변환하였는데, 이는 메모리 사용량을 줄이기 위한 것이기도 하지만, 뒤이어 사용할 mlxtend 라이브러리의 apriori 함수가 0/1 또는 True/False 형태의 입력을 전제로 하기 때문이기도 하다. 이렇게 만든 수평 테이블은 DMA_project2_team10_part1_horizontal.pkl 로 저장하였다.

(2) 빈발 아이템셋 및 연관 규칙 도출

생성한 수평 테이블에 대하여 apriori 알고리즘을 적용하되, 최소 지지도(min_support)를 0.005 로 설정하였다. 이는 전체 질문의 최소 0.5% 이상에서 함께 등장하는 태그 조합만을 빈발 아이템셋으로 인정한다는 의미이다. 그 결과 133 개의 빈발 아이템셋이 도출되었다. 이어서 향상도(lift)를 기준으로 최소 임계값을 2.0 으로 두고 연관 규칙을 추출한 결과, 최종적으로 28 개의 규칙을 얻었다.

```
frequent_itemsets = apriori(horizontal, min_support=0.005, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=2.0
)
```

도출된 규칙은 DMA_project2_team10_part1_association.pkl 로 저장하였다.

(3) lift 임계값 변화에 따른 규칙 수 분석

lift 임계값을 바꾸어 가며 추출되는 규칙 수가 어떻게 변하는지를 함께 확인하였다. 임계값을 1.0, 1.5, 2.0, 3.0, 5.0, 10.0 으로 높여 가자 규칙 수는 각각 40 개, 30 개, 28 개, 20 개, 12 개, 2 개로 감소하였다. 임계값이 1.0~1.5 로 낮을 때에는 regression 처럼 여러 통계 기법과 두루 함께 쓰이는 범용적인 태그 조합까지 폭넓게 검출되었으나, 임계값을 2.0 이상으로 높이자 arima 와 time-series 처럼 서로 대체하기 어려운 특화된 도메인 쌍의 비율이 높아졌다. 특히 임계값을 10.0 까지 올리면 repeated-measures 와 anova 처럼 거의 절대적인 상관을 가진 극소수 쌍만 남았다. 기본 임계값으로 채택한 2.0 은 범용 조합을 걸러내면서도 학술적·실무적으로 의미 있는 핵심 규칙들을 충분히 남기는 지점이라고 판단하였다.

(4) 도출된 규칙의 해석

향상도가 1 보다 큰 양의 연관 규칙 중 해석상 가치가 높은 것들을 살펴보았다. {repeated-measures} → {anova}는 향상도가 11.06 으로 가장 높았는데, 이는 동일한 개체를 반복 측정하는 실험 설계에서는 일반적인 분산분석이 아니라 반복측정 분산분석을 사용해야 하므로 두 개념이 거의 항상 함께 언급되기 때문으로 보인다. {arima} → {time-series}는 향상도 9.49, 신뢰도 0.61 로, ARIMA 가 시계열 분석의 대표적인 고전 모델인 만큼 ARIMA 질문을 올리는 사용자는 거의 시계열 태그를 함께 붙인다는 점이 드러난다. 이 밖에 {classification} → {machine-learning}(향상도 5.39), {probability} → {self-study}(향상도 3.78) 등도 도메인 상식과 잘 부합하는 규칙이었다.

반대로 향상도가 1 보다 작아 함께 등장하기를 꺼리는 음의 연관도 관찰되었다. {machine-learning} → {r}은 향상도가 0.53 에 불과하였는데, 이는 머신러닝 질문이 주로 Python 환경에서 이루어지는 반면 R 언어는 정통 통계 모델링이나 추론 중심 질문에 밀착되어 있어, 두 태그가 함께 붙을 확률이 서로 독립일 때의 기대치의 절반 수준으로 떨어지기 때문으로 해석된다. {machine-learning} → {regression} 역시 향상도가 0.70 으로, 작성자들이 추론 중심의 질문(regression)과 예측 중심의 질문(machine-learning)을 구분하여 태깅하는 경향을 보여 주었다.

3. PART II. 문서 검색 엔진

PART II에서는 통계·머신러닝 학술 논문 2,772 개로 이루어진 문서 집합(document.txt)에 대하여, 80 개의 자연어 질의어 각각에 대해 관련 문서를 관련도가 높은 순서로 반환하는 검색 엔진을 Whoosh 라이브러리 위에서 구축하였다. 성능은 evaluate.py 가 자동으로 산출하는 BPREF 값으로 측정되며, 각 질의어마다 정답 관련 문서가 정확히 15 개씩 존재한다. 우리의 최종 구성은 BPREF 0.3155 를 달성하였다.

(1) 코드 구조

검색 엔진의 코드는 역할에 따라 네 개의 모듈로 나누어 작성하였다. se_analyzer.py 는 질의어와 문서를 같은 어휘 공간으로 정규화하기 위한 형태소 분석기를 정의하는 모듈로, 토큰나이저와 소문자 변환, 불용어 제거에 이어 NLTK 의 Porter Stemmer 로 어간을 추출하는 분석기(get_porter_analyzer())를 제공한다. make_index.py 는 이 분석기를 사용하여 각 문서를 docID, title, contents 세 필드로 색인하는 모듈이며, 색인은 임시 폴더에 먼저 빌드한 뒤 완성되면 index 폴더로 원자적으로 옮기도록 하여 빌드 도중 오류가 나도 기존 인덱스가 손상되지 않도록 하였다.

CustomScoring.py 는 검색 점수를 매기는 모듈이다. Whoosh 가 제공하는 스코어링 기반 클래스 구조는 그대로 두고, 실제 점수 공식만을 intappscorer()라는 함수 하나로 분리해 두어 공식이나 가중치를 바꾸어 가며 실험하기 쉽도록 하였다. 마지막으로

QueryResult.py 는 실제 검색을 수행하는 모듈로, 질의어를 받아 토큰화하고 형태소로 정규화한 뒤 질의 객체를 만들어 1 차 검색을 수행하고, 검색된 문서를 다시 정렬(re-ranking)하여 최종 순위를 반환하는 getSearchEngineResult() 함수를 담고 있다. 전체 흐름은 질의어를 정규화하여 질의 객체를 만들고, 이를 점수 모델로 1 차 검색한 다음, 문서의 통계적 특성을 이용해 2 차로 재정렬하여 순위를 내보내는 순서이다.

(2) 실험 방법론

검색 품질을 결정하는 요소들, 즉 점수 공식과 IDF 지수, 질의어를 어떻게 구성하는지, 그리고 검색 후 어떤 신호로 재정렬하는지는 서로 영향을 크게 주고받는다. 따라서 어느 한 가지만 바꾸어 보고 결론을 내리기보다, 여러 후보 설정을 같은 조건에서 정량적으로 비교할 수 있는 실험 환경을 먼저 만들고 그 위에서 값을 탐색하기로 하였다.

문제는 `evaluate.py` 를 매번 그대로 실행하면 인덱스를 다시 열고 2,772 개 문서의 형태소를 다시 캐싱하는 데에만 15 초가량이 걸려, 설정 하나를 바꿔 볼 때마다 같은 준비 작업이 반복된다는 점이였다. 이에 인덱스와 형태소 캐시는 프로세스당 한 번만 적재하도록 분리하고, 점수 모델과 질의어 생성 방식, 재정렬 함수를 인자로 받아 BPREF 만 돌려주는 평가 함수를 따로 두었다. 이렇게 하자 설정 하나를 평가하는 데 걸리는 시간이 15 초에서 1~3 초로 줄어, TF 정규화 계수와 문서 길이 계수, IDF 지수 등을 넓게 바꾸어 가며 비교하는 것이 가능해졌다. 또한 질의어 생성 방식과 재정렬 방식을 독립 함수로 분리해 둔 덕분에, 각 요소를 하나씩만 켜고 끄면서 그 기여도를 따로 측정할 수 있었다. 성능이 낮게 나오는 질의어에 대해서는 그 질의어의 BPREF 와 recall 을 따로 출력하여, 관련 문서가 아예 검색되지 않는 것인지 아니면 검색은 되었으나 순위에서 밀린 것인지를 구분하여 원인을 추적하였다.

(3) 점수 모델의 결정

처음에는 기본 제공된 BM25F 모델을 기준으로 삼고, TF 정규화 계수 K_1 과 문서 길이 계수 B 를 조정해 가며 점수를 높여 보려 하였다. 그런데 의외로 TF 를 반영하는 BM25 계열 모델은 BPREF 가 0.28 수준에 머물러 오히려 기본보다 낮아졌고, K_1 을 0 에 가깝게 하여 단어의 등장 횟수를 무시하고 등장 여부만 보도록 할수록 점수가 올라갔다. 이는 학술 논문에서는 같은 키워드가 한 문서에 여러 번 등장한다고 해서 그 문서가 더 관련 있는 것이 아니며, 오히려 분량이 긴 문서에 점수가 물리는 잡음으로 작용한다는 것을 의미하였다.

이 관찰에 따라 TF 와 문서 길이 정규화를 모두 제거하고, 질의어 단어가 문서에 등장했는지 여부만을 IDF 가중치로 채점하는 단순한 모델을 채택하였다. 다만 희소한, 즉 더 특징적인 단어를 매칭한 문서를 더 강하게 끌어올리기 위하여 IDF 를 그대로 쓰지 않고 거듭제곱하여 사용하였는데, 지수를 1.0 부터 3.5 까지 바꾸어 본 결과 1.5 부근이 가장 좋았다. 지수가 지나치게 크면 희귀어 하나만 매칭한 문서가 과대평가되는 부작용이 있었다. 결국 점수 공식은 다음과 같이 매우 단순한 형태가 되었다.

```
def intappscorer(tf, idf, cf, qf, dc, fl, avgfl, param):
    if tf == 0:
        return 0.0
    return idf ** param    # param = 1.5
```

즉 문서의 점수는 질의어 중 그 문서에 등장한 단어들의 IDF 를 1.5 제곱하여 더한 값이 된다. 이 변경 하나만으로 BPREF 가 0.2497 에서 0.2993 으로 올라, 단일 변경으로는 가장 큰 효과를 보였다.

(4) 질의어의 구성

다음으로 질의어를 어떻게 검색 질의로 바꿀지를 실험하였다. 여러 방식을 비교한 결과, 질의어 전체를 하나의 구(phrase)로 묶되 단어 사이의 간격을 8 까지 허용하는 느슨한 근접 매칭과, 개별 단어들을 OR 로 묶은 항을 함께 사용하는 방식이 가장 안정적이었다. 근접 매칭의 허용 간격은 2, 4, 8, 15 로 바꾸어 보았는데 8 이 가장 좋았고, 간격을 2 로 지나치게 좁히면 어순이 조금 다른 유효한 문서를 놓쳐 점수가 떨어졌다. 개별 단어에는 단어 길이에 기반한 가중치를 부여하였고, 여러 질의어 단어가 함께 매칭된 문서에 약간의 보너스를 주기 위해 OR 그룹에 0.2 의 계수를 두었다. 이 계수는 0.0 에서 0.5 사이에서 결과가 거의 변하지 않아 보수적으로 0.2 를 택하였다. 예를 들어 “kernel methods support vector machine”이라는 질의어는 다음과 같은 형태의 질의 문자열로 변환된다.

```
"kernel method support vector machin"~8^10.0  
OR (kernel^2.7 methods^3.0 support^2.9 vector^2.6 machine^3.0)
```

이 과정에서 인접한 두 단어 쌍을 모두 강한 근접 구로 추가하거나, 약어를 자동으로 확장하거나, 제목을 별도 필드로 두어 부스팅하는 방법도 시도해 보았으나 모두 BPREF 가 오히려 떨어져 채택하지 않았다. 질의어 구성을 위 방식으로 바꾼 뒤 BPREF 는 0.2993 에서 0.3135 로 올랐다.

(5) 검색 결과의 재정렬

질의어별 진단을 해 보니, BPREF 가 0 인 질의어들은 관련 문서가 아예 검색되지 않은 것이 아니라 검색은 되었으나 무관한 문서들 아래로 밀려나 있는 경우가 많았다. 원인을 살펴보니, “process”나 “theory”처럼 흔한 단어를 여러 개 매칭한 무관한 문서가, 정작 희귀하고 핵심적인 단어를 매칭한 관련 문서보다 단순 점수 합에서 앞서기 때문이었다.

그래서 검색 상위 문서들을 대상으로 관련 문서와 무관 문서를 구분 짓는 특징이 무엇인지 정량적으로 비교해 보았다. 그 결과 질의어 단어 중 문서에 실제로 매칭된 단어들의 IDF 합이 전체 질의어 IDF 합에서 차지하는 비율, 즉 IDF 로 가중한 커버리지가 가장 강한 판별

신호였다. 단순히 매칭된 단어의 개수나 본문에서의 근접도는 무관 문서에서도 흔하게 나타나 판별력이 약하였고, 실제로 이들을 재정렬에 넣으면 오히려 BPREF 가 떨어졌다. 이에 1 차 점수에 IDF 가중 커버리지의 공급을 더한 항과, 질의어 구가 문서 제목에 그대로 포함되는지를 보는 제목 일치 항을 곱하여 최종 점수를 산출하였다. 구체적으로는 1 차 점수에 $(1 + \text{커버리지의 공급})$ 과 $(1 + 0.5 \times \text{제목 일치 여부})$ 를 곱하는 형태이며, 이로써 희귀한 핵심어를 두루 포함하면서 제목까지 질의어와 일치하는 문서가 상위로 올라오게 된다. 한편 document.txt 를 읽어 오지 못하는 경우에는 이 재정렬 단계를 건너뛰고 1 차 검색 결과를 그대로 반환하도록 예외 처리를 두어, 파일이 없는 환경에서도 검색 자체는 동작하도록 하였다. 재정렬까지 적용한 결과 BPREF 는 0.3135 에서 0.3155 로 올랐다.

(6) 성능 평가 및 한계 분석

지금까지의 개선을 정리하면, 기본 BM25F 와 단순 OR 질의로 측정한 baseline BPREF 가 0.2497 이었던 것을, TF 를 제거한 binary IDF 모델로 바꾸어 0.2993 으로, 구절 근접과 길이 가중 OR 질의로 0.3135 로, 마지막으로 IDF 가중 커버리지와 제목 일치 재정렬로 0.3155 까지 끌어올렸다. baseline 대비 0.0658 의 향상이며, 이 가운데 가장 큰 몫은 TF 를 제거한 점수 모델 변경(0.0496)이 차지하였다. 점수 환산식 $(BPREF - 0.2497) \times 300$ 을 적용하면 최종 점수는 약 19.7 점에 해당한다.

추가 개선의 한계도 함께 따져 보았다. 정답으로 주어진 관련 문서들 가운데 질의어와 형태소를 하나라도 공유하는 문서는 약 67.7%에 불과하였다. 즉 관련 문서의 약 3 분의 1 은 질의어와 표면적인 단어가 전혀 겹치지 않는 의미적 관련 문서로, 단어 일치에 기반한 어휘적 검색만으로는 애초에 찾아낼 수 없는 것들이다. 이는 어휘적 검색이 가질 수 있는 구조적 상한을 형성하며, 우리가 도달한 0.3155 는 허용된 도구(Whoosh 와 NLTK) 안에서 이 상한의 상당 부분을 어휘 신호만으로 달성한 결과라고 본다. 그 이상은 문장 임베딩과 같은 의미 기반 검색을 요구하는데, 이는 별도 라이브러리의 사전 승인이 필요한 범위이므로 본 보고서의 범위를 벗어난다.

마지막으로, 그럴듯해 보였으나 실제로는 효과가 없어 배제한 방법들을 기록해 둔다. TF 를 반영하는 BM25 계열 점수(약 0.28), 상위 문서를 이용한 유사 적합성 피드백(약 0.25~0.27), 인접 2-gram 의 강한 근접 부스팅(약 0.27~0.29), 약어 자동 확장, 제목을 별도 IDF 로 분리한 멀티필드 부스팅(약 0.21), Lancaster 등 공격적인 어간 추출(약 0.28~0.30), 그리고

커버리지를 1 순위로 두는 정렬(약 0.31)은 모두 BPREF 를 떨어뜨리거나 개선이 없었다. 이 과정을 통해, 이 문제에서는 정교해 보이는 기법을 더하는 것보다 TF 를 덜어낸 단순하고 견고한 점수 모델과 IDF 가중 커버리지 재정렬이 더 효과적이라는 점, 그리고 적지 않은 기법이 오히려 잡음을 더한다는 점을 정량적으로 확인할 수 있었다.

4. PART III. 문서 분류

PART III 에서는 네 개 학술 저널(AnnStat, Biometrika, JASA, JMLR)의 논문 본문을 대상으로, 주어진 논문이 어느 저널의 것인지를 분류하는 모델을 구축하였다. 학습 데이터는 저널당 200 개씩 총 800 개, 평가 데이터는 200 개이며, 코드는 CL/clasification.py 에서 데이터를 적재하고 두 종류의 분류기를 학습·평가·저장하도록 작성하였다. load_files 로 text/train 과 text/test 폴더 구조를 읽어 폴더 이름을 그대로 클래스 라벨로 사용하였다.

(1) 텍스트 피쳐 구성

이 네 저널은 모두 통계·머신러닝 분야에 속해 어휘가 상당 부분 겹치므로, 단어 단위의 정보만으로는 저널을 구분하기가 쉽지 않았다. 그래서 단어 단위 TF-IDF 와 문자 단위 TF-IDF 를 FeatureUnion 으로 결합하여, 단어 수준의 주제 정보와 더불어 저널마다 다른 표기·문체의 미세한 차이까지 함께 포착하도록 하였다. 단어 단위는 1~3-gram 을, 문자 단위는 단어 경계를 고려하는 char_wb 방식으로 짧은 n-gram 을 사용하였으며, 두 분류기에 대해 최소 문서 빈도와 가중치를 달리 두었다. 나이브 베이즈에서는 단어와 문자 피쳐의 가중치를 1.0 으로 동일하게 두었고, SVM 에서는 문자 피쳐가 저널 고유의 문체를 더 안정적으로 잡아내어 문자 가중치를 1.25 로 약간 높게 두었다.

여기서 한 가지 주의한 점은, 평가 데이터가 200 개로 작아 그 정확도만 보고 설정을 고르면 평가 데이터에 과적합될 위험이 있다는 것이다. 그래서 설정 탐색의 기준을 평가 정확도가 아니라 학습 데이터 800 개에 대한 5 겹 교차검증 정확도로 삼았다. 이 기준으로 SVM 의 문자 n-gram 범위와 가중치, 정규화 계수를 다시 탐색한 결과, 문자 2~4-gram 에 단어 대 문자 가중치 1.0 대 1.25, 정규화 계수 C 를 1.0 으로 둔 조합이 교차검증 정확도를 0.761 에서 0.789 로 높여, 이를 최종 설정으로 채택하였다.

(2) 분류 모델과 튜닝

첫 번째 모델은 FeatureUnion 위에 ComplementNB 를 올린 나이브 베이즈 모델이다. 처음에는 일반적인 MultinomialNB 를 사용했으나, 클래스 사이에 어휘가 많이 겹치는 이 데이터에서는 성능이 좋지 않았다. ComplementNB 는 클래스 불균형과 어휘 겹침에 더 강건하게 동작하므로 이를 채택하였고, 평활 계수 알파는 0.03 으로 작게 두어 FeatureUnion 으로 차원이 커진 환경에서도 확률 추정이 한쪽으로 쏠리지 않도록 하였다.

```
clf_nb = Pipeline([("tfidf", FeatureUnion([... word ..., ... char ...])),
                   ("clf", ComplementNB(alpha=0.03))])
```

두 번째 모델은 같은 FeatureUnion 위에 LinearSVC 를 올린 선형 SVM 이다. 단어와 문자 n-gram 을 합치면 피쳐 차원이 수만 차원으로 커지고 데이터가 희소해지는데, 이런 고차원 희소 데이터에서는 마진을 최대화하는 선형 SVM 의 일반화 성능이 좋다고 보았다. 교차검증 결과 짧은 문자 2~4-gram 이 저널 고유의 표기 습관을 잘 잡아냈고, 정규화 계수 C 를 1.0 으로 낮추었을 때 과적합이 완화되어 교차검증 정확도가 가장 높았다.

(3) 평가 결과

평가 데이터 200 개에 대한 최종 정확도는 나이브 베이즈가 76.5%(153/200), SVM 이 79.0%(158/200)였다. 두 모델의 분류 리포트는 다음과 같다.

[Naive Bayes]		precision	recall	f1-score	support
	AnnStat	0.65	0.80	0.71	50
	Biometrika	0.76	0.70	0.73	50
	JASA	0.81	0.78	0.80	50
	JMLR	0.89	0.78	0.83	50
	accuracy			0.77	200
[SVM]		precision	recall	f1-score	support
	AnnStat	0.69	0.84	0.76	50
	Biometrika	0.79	0.76	0.78	50
	JASA	0.87	0.78	0.82	50
	JMLR	0.85	0.78	0.81	50
	accuracy			0.79	200

단어와 문자 n-gram 을 FeatureUnion 으로 결합하고 하이퍼파라미터를 다시 탐색한 결과, 나이브 베이즈는 기존 69.0%에서 76.5%로, SVM 은 78.0%에서 79.0%로 향상되었다. 특히 ComplementNB 로 바꾸면서 AnnStat 과 Biometrika 사이의 혼동 오분류가 기존 17 건에서

11 건으로 줄어든 점이 두드러졌다. 다만 평가 데이터에서의 최종 정확도 79.0%는 고전 통계 저널인 AnnStat, Biometrika, JASA 사이의 주제가 본래 크게 겹친다는 데이터 자체의 한계에 가까워, 평가 정확도를 더 끌어올리기보다는 교차검증 기준으로 모델의 일반화 견고성을 높이는 방향으로 마무리하였다.

5. 결론

본 프로젝트에서는 세 가지 서로 다른 데이터마이닝 과업을 각각 독립된 모듈로 구현하고, 단순히 결과를 내는 데 그치지 않고 어떤 시도가 실제로 효과가 있었는지를 실험으로 확인하고자 하였다. PART I에서는 연관 분석을 통해 도메인 상식과 부합하는 28 개의 태그 연관 규칙을 도출하고 양·음의 연관을 해석하였다. PART II에서는 빠른 평가 환경을 먼저 구축한 뒤 점수 모델과 질의어 구성, 재정렬을 차례로 개선하여 BPREF 를 0.2497 에서 0.3155 까지 높였으며, 그 핵심이 TF 를 제거한 단순한 binary IDF 모델에 있었음을 확인하였다. PART III에서는 단어와 문자 n-gram 을 결합하고 교차검증을 기준으로 설정을 탐색하여 나이브 베이즈 76.5%, SVM 79.0%의 정확도를 얻었다. 세 파트에 공통적으로, 그럴듯해 보이는 기법을 무작정 더하기보다 정량적인 비교를 통해 실제로 효과 있는 방법과 잡음을 더하는 방법을 가려낸 것이 이번 프로젝트에서 얻은 가장 중요한 교훈이었다.

각 파트의 결과는 폴더 안에서 `python part1.py`, `python evaluate.py`, `python clasification.py` 를 실행하여 그대로 재현할 수 있다.